Constraint-Aware Minority Oversampling via Relaxed Functional Dependencies [Supplementary Materials]

Mary Cerullo
Liris, CNRS
Lyon, France
mary.cerullo@liris.cnrs.fr

Stefano Cirillo
University of Salerno
Fisciano, Salerno, Italy
scirillo@unisa.it

Gianpaolo Iuliano
LIRIS, CNRS
Lyon, France
gianpaolo.iuliano@cnrs.fr

Giuseppe Polese
University of Salerno
Fisciano, Salerno, Italy
gpolese@unisa.it

Giandomenico Solimando
University of Salerno
Fisciano, Salerno, Italy
gsolimando@unisa.it

1 ADDITIONAL ALGORITHMS

In this section, we report the pseudocode of the procedures omitted from the main text: the complete training loop of SyRFDRes (Algorithm 3) and the history-aware filtering used by the validator in strict mode (Algorithm 4).

1.1 Training Procedure

Algorithm 3 details the procedure behind the two-phase training schedule (Sec. 4.3). The procedure starts with the initialization of the TabResNet generator, the optimizer with its cosine-restart scheduler, and the rolling buffer (Lines 1–2). Then, it computes the per-attribute normalization statistics $\mathbf{a}$ and $\mathbf{r}$ from D^+ and prepares two reference views of the minority partition: the raw tensor $\mathbf{X}_{\text{real}}$ (Line 4), which $\mathcal{L}_{\text{RFD}}$ consumes in original-domain units so that the thresholds in Φ retain their physical meaning, and its normalized version $\tilde{\mathbf{X}}_{\text{real}}$, which is applied for computing the Silhouette and DBI monitoring statistics. At each iteration, the algorithm applies the sigmoid σ to the raw generator output, obtaining the normalized batch $\hat{\mathbf{x}}_n \in [0,1]^m$, and denormalized it into $\hat{\mathbf{x}}_d$ (Lines 10–11): the former feeds the geometric terms, the latter feeds $\mathcal{L}_{\text{MMD}}$ and $\mathcal{L}_{\text{RFD}}$. The algorithm then samples a real mini-batch with replacement, normalizes it, and pushes the generated batch into the buffer Q. During the warmup phase, the procedure assembles the objective of Eq. (11) (Lines 16–17); afterwards, it switches to the full objective of Eq. (5) and logs the monitoring statistics over Q and $\tilde{\mathbf{X}}_{\text{real}}$ (Lines 18–22). After the backpropagation of the loss, clipping the gradient norms, and stepping the optimizer and the scheduler, the procedure evaluates the early-stopping criterion (Lines 24–29) only in Phase 2 and only once Q has reached its minimum fill $q_{\min}$, so that the decision to stop rests on stable estimates rather than on single-batch noise. Finally, the procedure returns the generator G_θ.

Algorithm 3 SyRFDRes Training

Require: minority partition D^+, RFDs Σ, thresholds Φ, iterations T, warmup T_w, batch size B, temperature τ, projections P, weights $\lambda_{\text{fid}}, \lambda_{\text{rfd}}, \lambda_{\text{comp}}, w_{\text{swd}}, w_{\text{cov}}, w_{\text{div}}$, patience $p_{\max}$, tolerance δ

1: Initialise G_θ (TabResNet), Adam optimiser, cosine-restart scheduler
2: Initialise rolling buffer Q with capacity $q_{\max}$, min fill $q_{\min}$
3: $\mathbf{a} \leftarrow \min(D^+)$, $\mathbf{r} \leftarrow \max(D^+) - \mathbf{a}$ ▷ per-attribute min and range
4: $\mathbf{X}_{\text{real}} \leftarrow D^+$ ▷ raw reference tensor, original domain
5: $\tilde{\mathbf{X}}_{\text{real}} \leftarrow \text{normalise}(D^+, \mathbf{a}, \mathbf{r})$ ▷ normalized reference, Silhouette/DBI monitoring only
6: $\ell^* \leftarrow +\infty$, $p \leftarrow 0$
7: **for** $e = 1$ **to** T **do**
8: $\quad \mathbf{Z} \sim \mathcal{N}(0, I_{d_z})^B$
9: $\quad \hat{\mathbf{x}}_{\text{raw}} \leftarrow G_\theta(\mathbf{Z})$
10: $\quad \hat{\mathbf{x}}_n \leftarrow \sigma(\hat{\mathbf{x}}_{\text{raw}})$ ▷ single sigmoid: output in $[0,1]^m$
11: $\quad \hat{\mathbf{x}}_d \leftarrow \hat{\mathbf{x}}_n \odot \mathbf{r} + \mathbf{a}$ ▷ denormalize to original domain
12: $\quad$ Sample mini-batch $\mathbf{x}_B \sim D^+$; $\mathbf{x}_n \leftarrow \text{clamp}((\mathbf{x}_B - \mathbf{a})/\mathbf{r}, 0, 1)$
13: $\quad$ Push $\hat{\mathbf{x}}_n$ into buffer Q
14: $\quad \mathcal{L}_{\text{MMD}} \leftarrow \text{MMD}(\hat{\mathbf{x}}_d, \mathbf{x}_B, \mathbf{a}, \mathbf{r})$ ▷ Eq. (7)
15: $\quad \mathcal{L}_{\text{RFD}} \leftarrow \text{SoftRFD}(\hat{\mathbf{x}}_d, \mathbf{X}_{\text{real}}, \Sigma, \Phi, \tau)$ ▷ Eq. (8), full raw reference
16: $\quad$ **if** $e \le T_w$ **then** ▷ Phase 1 - Warmup
17: $\quad\quad \mathcal{L} \leftarrow \lambda_{\text{fid}}\,\mathcal{L}_{\text{MMD}} + \lambda_{\text{rfd}}\,\mathcal{L}_{\text{RFD}}$ ▷ Eq. (11)
18: $\quad$ **else** ▷ Phase 2 - Refinement
19: $\quad\quad \mathcal{L}_{\text{SWD}} \leftarrow \text{SWD}(\hat{\mathbf{x}}_n, \mathbf{x}_n, P)$ ▷ Eq. (9)
20: $\quad\quad \mathcal{L}_{\text{cov}} \leftarrow \text{CovLoss}(\hat{\mathbf{x}}_n, \mathbf{x}_n, w_{\text{div}})$ ▷ Eq. (10)
21: $\quad\quad \mathcal{L}_{\text{comp}} \leftarrow w_{\text{swd}}\,\mathcal{L}_{\text{SWD}} + w_{\text{cov}}\,\mathcal{L}_{\text{cov}}$ ▷ Silhouette/DBI logged using $Q, \tilde{\mathbf{X}}_{\text{real}}$
22: $\quad\quad \mathcal{L} \leftarrow \lambda_{\text{fid}}\,\mathcal{L}_{\text{MMD}} + \lambda_{\text{rfd}}\,\mathcal{L}_{\text{RFD}} + \lambda_{\text{comp}}\,\mathcal{L}_{\text{comp}}$ ▷ Eq. (5)
23: $\quad \mathcal{L}.\text{backward}()$; $\text{clip}(\nabla_\theta, 1.0)$; Adam step; scheduler step
24: $\quad$ **if** $e > T_w$ **and** $|Q| \ge q_{\min}$ **then** ▷ early stopping on Phase 2 only
25: $\quad\quad$ **if** $\mathcal{L}_{\text{comp}} < \ell^* - \delta$ **then**
26: $\quad\quad\quad \ell^* \leftarrow \mathcal{L}_{\text{comp}}$; $p \leftarrow 0$
27: $\quad\quad$ **else**
28: $\quad\quad\quad p \leftarrow p + 1$
29: $\quad\quad\quad$ **if** $p \ge p_{\max}$ **then break**
30: **return** G_θ

1.2 Strict-Mode Filtering

Algorithm 4 details the history-aware filtering that Algorithm 2 invokes in strict mode. The procedure first checks the whole candidate batch against the real data through VCheck (Line 1) and retains the survivors (Line 2). If the acceptance history $\mathcal{H}$ is non-empty, it runs a second batched check against $\mathcal{H}$, discarding the candidates that would conflict with tuples accepted in previous rounds (Lines 3–5). Both checks reuse VCheck (Algorithm 1) and therefore run as broadcast operations. Finally, the algorithm performs the sequential intra-batch pass (Lines 6–9): it visits the candidates in generation order, checks each one against the history accumulated so far and, if the candidate is valid, appends it to both the accepted set $\mathcal{A}$ and the history $\mathcal{H}$ before evaluating the next one (Line 9). The procedure returns the accepted tuples, i.e., those to be inserted in the augmented set, together with the updated history. This ordering is what makes the mutual consistency of Theorem 4.1 hold: the procedure never accepts two tuples without having checked them against each other.

Algorithm 4 History-Aware Strict Filtering (StrictFilter)

Require: candidates $\hat{\mathbf{X}} \in \mathbb{R}^{b\times m}$, minority partition D^+, history $\mathcal{H}$, RFDs Σ, thresholds Φ
Ensure: accepted tuples $\mathcal{A}$, updated history $\mathcal{H}$

$\mathbf{m}_v \leftarrow \text{VCheck}(\hat{\mathbf{X}}, D^+, \Sigma, \Phi)$ ▷ batch vs. real data, broadcast
$\mathbf{C} \leftarrow \hat{\mathbf{X}}[\mathbf{m}_v]$
if $\mathcal{H} \neq \emptyset$ **then**
  $\mathbf{m}_h \leftarrow \text{VCheck}(\mathbf{C}, \mathcal{H}, \Sigma, \Phi)$ ▷ batch vs. accepted history, broadcast
  $\mathbf{C} \leftarrow \mathbf{C}[\mathbf{m}_h]$
$\mathcal{A} \leftarrow \emptyset$
for each candidate $\mathbf{c} \in \mathbf{C}$, in generation order **do** ▷ sequential intra-batch pass
  if $\mathcal{H} = \emptyset$ **or** $\text{VCheck}(\{\mathbf{c}\}, \mathcal{H}, \Sigma, \Phi)$ **then**
    $\mathcal{A} \leftarrow \mathcal{A} \cup \{\mathbf{c}\}$; $\mathcal{H} \leftarrow \mathcal{H} \cup \{\mathbf{c}\}$ ▷ accepted before next candidate is evaluated
return $\mathcal{A}, \mathcal{H}$

2 EVALUATION METRICS

Let $R = \{\mathbf{x}^{(1)}, \ldots, \mathbf{x}^{(n_r)}\}$ be the real minority tuples and $G = \{\hat{\mathbf{x}}^{(1)}, \ldots, \hat{\mathbf{x}}^{(n_g)}\}$ the generated ones, defined over m common numerical attributes. Attribute-wise metrics are computed per attribute A and averaged over the m attributes. Histogram-based metrics discretize each attribute into b equal-width bins over the combined range of R and G, with add-one smoothing; p_A and q_A denote the resulting generated and real bin frequencies. Spatial metrics operate on min–max normalized data, with the scaler fitted on R only, generated values clipped to $[0, 1]$, and zero-variance attributes excluded. We denote by $\mu_{\cdot,A}$, $s_{\cdot,A}$, and $v_{\cdot,A}$ the mean, standard deviation, and variance of attribute A in the given set, and by $F_{\cdot,A}$ its Cumulative Distribution Function (CDF).

2.1 Fidelity Metrics

Kullback–Leibler divergence [32]. The average divergence of the generated histograms from the real ones ($b = 30$):

$$\mathrm{KL} = \frac{1}{m}\sum_{A=1}^{m}\sum_{i=1}^{b} p_A(i)\,\log\frac{p_A(i)}{q_A(i)}. \tag{15}$$

It penalizes generated mass placed where the real distribution has little support: values close to zero indicate that the generated histograms closely track the real ones, while large values reveal attributes where the generator over-populates regions the real data rarely visits.

Jensen–Shannon divergence [40]. The symmetrized, bounded counterpart of KL, with $M_A = \frac{1}{2}(p_A + q_A)$:

$$\mathrm{JS} = \frac{1}{m}\sum_{A=1}^{m}\frac{1}{2}\,D_{\mathrm{KL}}(p_A \,\|\, M_A) + \frac{1}{2}\,D_{\mathrm{KL}}(q_A \,\|\, M_A). \tag{16}$$

Unlike KL, it is finite even on disjoint supports: values near zero indicate near-identical generated and real histograms, whereas values approaching its upper bound ($\log 2$, attribute-wise) indicate that the two distributions barely overlap.

Kolmogorov–Smirnov statistic [5, 51]. The maximum gap between the empirical CDFs:

$$\mathrm{KS} = \frac{1}{m}\sum_{A=1}^{m}\sup_{x}\left|F_{G,A}(x) - F_{R,A}(x)\right|. \tag{17}$$

It captures the worst possible distributional mismatch per attribute: values close to zero mean that the generated and real distributions never deviate much, while values close to one indicate a point where almost none of the generated mass matches the real one.

χ^2 statistic [2]. The Pearson statistic between generated and real bin counts ($b = 20$), with generated counts rescaled to the real total and the result normalized by n_g:

$$\chi^2 = \frac{1}{m}\sum_{A=1}^{m}\frac{1}{n_g}\sum_{i=1}^{b}\frac{\left(\tilde{o}_A(i) - e_A(i)\right)^2}{e_A(i)}, \tag{18}$$

where e_A are the real counts and $\tilde{o}_A$ the rescaled generated counts. Values near zero indicate that generated and real bin counts agree closely, while large values flag bins where the generated frequency deviates sharply from what the real histogram would predict.

Wasserstein distance [18, 46]. The mean absolute gap between quantile functions, evaluated on a grid of 100 equally spaced levels $u_t \in [0, 1]$:

$$\mathrm{WD} = \frac{1}{m}\sum_{A=1}^{m}\frac{1}{100}\sum_{t=1}^{100}\left|F^{-1}_{R,A}(u_t) - F^{-1}_{G,A}(u_t)\right|. \tag{19}$$

Being quantile-based, it remains informative on disjoint supports and is sensitive to tails: values close to zero mean the generated and real quantile functions nearly coincide at every level, while large values indicate that the generated attribute is systematically shifted or stretched relative to the real one, including in its extremes.

PCA Wasserstein distance [46]. The metric computes the same quantile distance as WD with the PCA fitted on normalized real data and applied to the generated data:

$$\mathrm{WD}_{\mathrm{PCA}} = \frac{1}{3}\sum_{d=1}^{3}\frac{1}{100}\sum_{t=1}^{100}\left|F^{-1}_{R,d}(u_t) - F^{-1}_{G,d}(u_t)\right|. \tag{20}$$

Unlike WD, which treats each dimension independently and is blind to inter-attribute correlations, $\mathrm{WD}_{\mathrm{PCA}}$ compares the joint distributions along their directions of maximal variance, capturing whether the generated set preserves the correlation structure of the real data rather than just its marginals: values near zero indicate that the dominant modes of joint variation are preserved, while large values reveal that the generated tuples distort the correlations among attributes even if individual marginals look correct.

Propensity score MSE (pMSE) [52]. A linear propensity model $\hat{\pi}$ is fitted by least squares to discriminate real from generated normalized tuples, with predictions clipped to $[0, 1]$:

$$\mathrm{pMSE} = \frac{1}{n_r + n_g}\sum_{i=1}^{n_r+n_g}\left(\hat{\pi}_i - c\right)^2, \qquad c = \frac{n_g}{n_r + n_g}. \tag{21}$$

If the two sets are indistinguishable, every prediction collapses to the base rate c and the metric approaches zero, meaning no linear model can tell generated and real tuples apart; larger values indicate

that real and generated tuples occupy identifiably different regions of the attribute space.

Maximum Mean Discrepancy [7]. The multi-scale RBF estimate of Eq. (7) with the kernel of Eq. (6), computed on normalized data. It compares the joint distributions through their kernel mean embeddings, capturing discrepancies in all moments: values near zero indicate that generated and real samples are statistically indistinguishable under the kernel embedding, while larger values indicate a growing mismatch between the two distributions as a whole, not just in their marginals.

Moment differences. The discrepancies of the first four standardized moments on normalized data:

$$\Delta_\mu = \frac{1}{m}\sum_A \frac{|\mu_{R,A} - \mu_{G,A}|}{s_{R,A}}, \qquad \Delta_v = \frac{1}{m}\sum_A \frac{|v_{R,A} - v_{G,A}|}{v_{R,A}}, \tag{22}$$

$$\Delta_{\text{skew}} = \frac{1}{m}\sum_A |\gamma_{R,A} - \gamma_{G,A}|, \tag{23}$$

where γ and κ denote skewness and excess kurtosis. They localize the mismatch on specific moments, i.e., location, spread, asymmetry, and tail weight: values near zero mean the generated attribute matches the real one on that specific moment, while large values pinpoint exactly which aspect of the distribution is misrepresented.

2.2 Diversity Metrics

Precision, Recall, Density, and Coverage [33, 42]. Let $r_k(\mathbf{x})$ be the distance from $\mathbf{x}$ to its k-th nearest neighbor within its own set ($k = 5$), and $\mathbb{1}[\cdot]$ the indicator function:

$$\text{Prec} = \frac{1}{n_g}\sum_j \mathbb{1}\Big[\exists\, i : \|\hat{\mathbf{x}}^{(j)} - \mathbf{x}^{(i)}\| \le r_k(\mathbf{x}^{(i)})\Big], \tag{24}$$

$$\text{Rec} = \frac{1}{n_r}\sum_i \mathbb{1}\Big[\exists\, j : \|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(j)}\| \le r_k(\hat{\mathbf{x}}^{(j)})\Big], \tag{25}$$

$$\text{Dens} = \frac{1}{n_g}\sum_j \frac{1}{k}\sum_i \mathbb{1}\Big[\|\hat{\mathbf{x}}^{(j)} - \mathbf{x}^{(i)}\| \le r_k(\mathbf{x}^{(i)})\Big], \tag{26}$$

$$\text{Cov} = \frac{1}{n_r}\sum_i \mathbb{1}\Big[\exists\, j : \|\mathbf{x}^{(i)} - \hat{\mathbf{x}}^{(j)}\| \le r_k(\mathbf{x}^{(i)})\Big]. \tag{27}$$

Precision measures the fraction of generated tuples lying on the real manifold; Recall and Coverage measure the fraction of real tuples reached by the generated set (with neighborhoods defined by the generated and the real set, respectively); Density refines Precision by counting how many real neighborhoods contain each generated tuple. For all four, values near one indicate that generated and real samples densely overlap in the corresponding sense (fidelity to the manifold for Precision/Density, support coverage for Recall/Coverage), while values near zero indicate that generated tuples fall outside the real manifold (low Precision/Density) or that large parts of the real support are never reached (low Recall/Coverage).

Silhouette score. Computed on $G \cup R$ with set membership as cluster label; for each point, a is the mean intra-cluster distance and b the mean distance to the other cluster:

$$\text{Sil} = \frac{1}{n_r + n_g}\sum_i \frac{b_i - a_i}{\max(a_i, b_i)}. \tag{28}$$

Values near zero indicate that generated and real samples are spatially intermixed—desirable in this context, where separability signals distributional mismatch; values approaching one indicate that the two sets form well-separated clusters, i.e., that generated tuples are easily distinguishable from real ones by their location alone.

Davies–Bouldin index [15]. On the same two-cluster labeling, with S_G, S_R the mean within-set distances to the respective centroids $\mathbf{c}_G, \mathbf{c}_R$:

$$\text{DBI} = \frac{S_G + S_R}{\|\mathbf{c}_G - \mathbf{c}_R\|_2}. \tag{29}$$

Contrary to its use in clustering, higher is better here: a large value means the two sets overlap far more than they separate, i.e., the generated and real centroids are close relative to each set's internal spread; a small value means the two centroids are far apart relative to that spread, indicating that generated and real tuples occupy visibly different regions.

Intra-compactness [61]. The average per-attribute variance of the generated set:

$$\mathcal{I}_{Com} = \frac{1}{m}\sum_{A=1}^{m} v_{G,A}. \tag{30}$$

Values close to zero reveal mode collapse, i.e., the generator produces nearly identical tuples; values comparable to the real variance indicate that the generated set spreads out as much as the real data, without necessarily implying that it spreads correctly.

Mean distance to closest record (mDCR) [62]. The average distance of each generated tuple from its nearest real tuple:

$$\text{mDCR} = \frac{1}{n_g}\sum_{j=1}^{n_g} \min_i \|\hat{\mathbf{x}}^{(j)} - \mathbf{x}^{(i)}\|_2. \tag{31}$$

Values near zero indicate near-duplication of real tuples, i.e., the generator is memorizing rather than generalizing; large values indicate drift off the real support, i.e., generated tuples that resemble no real tuple closely; intermediate values are desirable, as they indicate novel but plausible tuples.

Quantile-bin coverage. Each real attribute is partitioned into its decile bins. The metric is the fraction of these bins hit by at least one generated value, following the same support-coverage rationale as [21] but adapted to continuous attributes via quantile binning rather than categorical level counts:

$$\text{Cov}_q = \frac{1}{m}\sum_{A=1}^{m} \frac{\big|\{\, i : \exists\, j,\ \hat{x}_A^{(j)} \in \text{bin}_i \,\}\big|}{\#\text{bins}_A}. \tag{32}$$

It verifies that the generated values populate the full range of each real attribute, including low-density regions: values near 1 mean the generated set reaches every decile of the real distribution, including its sparsely populated tails, while low values mean entire regions of the real attribute ranges.

2.3 RFD Compliance Metrics

As in Section 5.1.3, S denotes the synthetic tuples of the augmented dataset and viol the predicate of Eq. (1); as a protocol note, original tuples replicated verbatim in the augmented files are excluded from S, since they are compliant by construction and would inflate the

scores of all methods. Tuple-level compliance instantiates Eq. (14) at the two scopes of Definitions 3.3 and 3.4:

$$\mathrm{TC}_{\mathrm{rel}} = \frac{\left|\{\, t \in S \;:\; \mathrm{viol}(t, t', \varphi) = 0 \;\; \forall\, t' \in D^{+},\; \varphi \in \Sigma \,\}\right|}{|S|}, \tag{33}$$

$$\mathrm{TC}_{\mathrm{str}} = \frac{\left|\{\, t \in S \;:\; \mathrm{viol}(t, t', \varphi) = 0 \;\; \forall\, t' \in D^{+} \cup S \setminus \{t\},\; \varphi \in \Sigma \,\}\right|}{|S|}. \tag{34}$$

Table 1 reports the detailed RFD tuple compliance scores for all methods across the three threshold levels $\varphi \in \{2, 4, 8\}$, expressed as mean μ and standard deviation σ over all datasets. The table complements Figure 4 by providing the exact numerical values underlying the visual comparison, allowing a fine-grained inspection of compliance at each threshold level and validation mode. For each method, the relaxed and strict compliance scores are reported separately, reflecting the two granularities at which dependency preservation is measured: relaxed compliance evaluates each synthetic tuple independently against D^{+}, while strict compliance additionally requires consistency among all synthetic tuples in the augmented set. The standard deviations highlight the dataset-dependent variability of compliance across methods: while SyRFDRes with fixed thresholds and matching validation mode achieves zero variance in strict mode, confirming that perfect compliance is guaranteed by construction, most baselines and LLM-based strategies exhibit high variance, indicating that their compliance is heavily influenced by the relational structure of the specific dataset being augmented.

Algorithm	$\varphi = 2$				$\varphi = 4$				$\varphi = 8$			
	Relaxed		Strict		Relaxed		Strict		Relaxed		Strict	
	mean	std	mean	std	mean	std	mean	std	mean	std	mean	std
Geometric methods												
SMOTE	0.973	0.082	0.891	0.292	0.979	0.101	0.951	0.197	0.999	0.007	0.991	0.043
SMOTE-CDNN	0.970	0.089	0.891	0.287	0.980	0.094	0.954	0.196	0.999	0.006	0.991	0.045
FSMOTE	0.932	0.140	0.905	0.194	0.978	0.100	0.971	0.136	0.998	0.008	0.996	0.021
Deep generative models												
CTABGAN	0.693	0.318	0.519	0.427	0.958	0.096	0.879	0.265	0.997	0.008	0.952	0.166
CTABGAN+	0.932	0.140	0.828	0.320	0.987	0.061	0.933	0.216	0.997	0.011	0.959	0.152
DDPM	0.883	0.269	0.759	0.376	0.954	0.189	0.921	0.271	0.964	0.184	0.961	0.196
GOGGLE	0.296	0.395	0.114	0.321	0.594	0.383	0.313	0.448	0.919	0.271	0.759	0.370
TVAE	0.709	0.288	0.514	0.424	0.900	0.170	0.849	0.240	0.990	0.044	0.956	0.149
CasTGAN	0.691	0.333	0.582	0.365	0.873	0.260	0.801	0.354	0.976	0.072	0.902	0.285
CUTS	0.927	0.155	0.725	0.397	0.987	0.061	0.879	0.323	0.999	0.004	0.923	0.272
PreFair	0.876	0.240	0.731	0.380	0.966	0.120	0.921	0.222	0.998	0.008	0.968	0.114
TabFairGAN	0.866	0.296	0.789	0.372	0.940	0.213	0.903	0.283	0.966	0.172	0.962	0.193
TabularARGN	0.951	0.153	0.905	0.269	0.972	0.101	0.913	0.259	0.988	0.049	0.950	0.193
LLM-based — Devstral-Small-24B												
Dev/conf	0.900	0.216	0.800	0.336	0.959	0.155	0.819	0.360	0.959	0.161	0.946	0.201
Dev/distr	0.865	0.262	0.718	0.408	0.924	0.228	0.809	0.365	0.933	0.229	0.885	0.320
Dev/dtree	0.840	0.271	0.657	0.419	0.910	0.243	0.818	0.362	0.915	0.253	0.884	0.325
Dev/ens	0.795	0.285	0.474	0.439	0.899	0.227	0.678	0.421	0.918	0.233	0.833	0.368
Dev/hier	0.921	0.156	0.774	0.362	0.958	0.133	0.841	0.326	0.952	0.168	0.912	0.274
Dev/react	0.955	0.090	0.792	0.347	0.974	0.066	0.836	0.326	0.992	0.033	0.922	0.272
Dev/schema	0.927	0.150	0.798	0.313	0.949	0.139	0.826	0.339	0.962	0.138	0.944	0.210
Dev/self	0.854	0.299	0.760	0.383	0.911	0.233	0.776	0.394	0.918	0.237	0.846	0.368
LLM-based — Gemma-3-12B												
G3/conf	0.856	0.295	0.605	0.438	0.894	0.234	0.721	0.437	0.915	0.224	0.795	0.400
G3/distr	0.838	0.307	0.665	0.431	0.881	0.266	0.766	0.417	0.906	0.248	0.824	0.375
G3/dtree	0.879	0.238	0.555	0.470	0.954	0.095	0.769	0.403	0.971	0.081	0.875	0.328
G3/ens	0.861	0.187	0.390	0.412	0.928	0.166	0.587	0.450	0.940	0.171	0.798	0.382
G3/hier	0.854	0.288	0.585	0.462	0.897	0.225	0.739	0.419	0.914	0.221	0.790	0.403
G3/react	0.857	0.279	0.592	0.442	0.909	0.226	0.790	0.374	0.920	0.225	0.839	0.360
G3/schema	0.840	0.302	0.542	0.448	0.896	0.244	0.764	0.412	0.905	0.238	0.790	0.404
G3/self	0.862	0.291	0.595	0.427	0.929	0.205	0.763	0.409	0.931	0.215	0.793	0.398
LLM-based — Gemma-4-31B												
G4/conf	0.914	0.226	0.769	0.403	0.935	0.211	0.818	0.352	0.958	0.195	0.918	0.271
G4/distr	0.919	0.195	0.764	0.381	0.937	0.177	0.842	0.340	0.960	0.136	0.913	0.272
G4/dtree	0.876	0.239	0.748	0.393	0.895	0.246	0.785	0.373	0.918	0.236	0.847	0.365
G4/ens	0.886	0.221	0.486	0.453	0.917	0.200	0.712	0.419	0.942	0.189	0.875	0.326
G4/hier	0.956	0.100	0.750	0.392	0.974	0.082	0.828	0.344	0.992	0.028	0.917	0.271
G4/react	0.955	0.132	0.834	0.353	0.980	0.083	0.857	0.309	0.996	0.015	0.960	0.192
G4/schema	0.932	0.144	0.742	0.411	0.954	0.125	0.808	0.360	0.988	0.039	0.916	0.266
G4/self	0.960	0.096	0.780	0.375	0.977	0.082	0.841	0.330	0.993	0.026	0.939	0.216
LLM-based — GPT-OSS-120B												
GPT120/conf	0.839	0.293	0.712	0.407	0.926	0.211	0.775	0.386	0.947	0.190	0.878	0.324
GPT120/distr	0.815	0.307	0.673	0.433	0.904	0.255	0.803	0.365	0.932	0.237	0.921	0.271
GPT120/dtree	0.806	0.291	0.493	0.426	0.888	0.223	0.655	0.427	0.929	0.214	0.791	0.402
GPT120/ens	0.780	0.303	0.388	0.426	0.886	0.224	0.531	0.449	0.911	0.235	0.725	0.450
GPT120/hier	0.854	0.270	0.697	0.403	0.910	0.251	0.825	0.331	0.930	0.236	0.891	0.308
GPT120/react	0.802	0.330	0.599	0.457	0.911	0.218	0.772	0.367	0.937	0.206	0.841	0.367
GPT120/schema	0.800	0.323	0.604	0.446	0.898	0.256	0.786	0.373	0.929	0.237	0.842	0.367
GPT120/self	0.809	0.312	0.668	0.408	0.910	0.218	0.797	0.339	0.941	0.201	0.883	0.325
LLM-based — GPT-OSS-20B												
GPT20/conf	0.808	0.299	0.552	0.438	0.921	0.185	0.703	0.428	0.937	0.184	0.807	0.401
GPT20/distr	0.848	0.257	0.557	0.463	0.932	0.183	0.698	0.453	0.950	0.166	0.770	0.424
GPT20/dtree	0.793	0.273	0.327	0.413	0.890	0.236	0.634	0.448	0.933	0.180	0.757	0.427
GPT20/ens	0.772	0.308	0.381	0.409	0.889	0.244	0.611	0.447	0.913	0.233	0.758	0.427
GPT20/hier	0.816	0.304	0.496	0.454	0.907	0.205	0.614	0.446	0.931	0.197	0.795	0.400
GPT20/react	0.807	0.296	0.421	0.430	0.907	0.199	0.632	0.446	0.931	0.195	0.806	0.401
GPT20/schema	0.793	0.299	0.502	0.451	0.892	0.233	0.672	0.451	0.927	0.207	0.760	0.426
GPT20/self	0.822	0.279	0.496	0.461	0.920	0.164	0.630	0.451	0.951	0.152	0.726	0.450
LLM-based — Zero-shot												
DeepSeek	0.957	0.126	0.888	0.277	0.976	0.097	0.927	0.256	0.972	0.126	0.940	0.215
LLaMA	0.663	0.375	0.298	0.433	0.730	0.349	0.432	0.466	0.806	0.334	0.576	0.489
SyRFDRes												
SyRFDRes φ=2/*rlx*	**1.000**	0.000	0.919	0.240	0.978	0.111	0.938	0.218	0.999	0.003	0.975	0.126
SyRFDRes φ=2/*str*	**1.000**	0.000	**1.000**	0.000	0.983	0.084	0.945	0.210	**1.000**	0.002	0.980	0.102
SyRFDRes φ=4/*rlx*	0.951	0.138	0.884	0.297	**1.000**	0.000	0.942	0.212	0.999	0.004	0.974	0.131
SyRFDRes φ=4/*str*	0.953	0.130	0.887	0.286	**1.000**	0.000	**1.000**	0.000	0.999	0.005	0.984	0.081
SyRFDRes φ=8/*rlx*	0.963	0.129	0.901	0.243	0.982	0.089	0.945	0.205	**1.000**	0.000	0.979	0.105
SyRFDRes φ=8/*str*	0.955	0.123	0.880	0.300	0.980	0.102	0.953	0.198	**1.000**	0.000	**1.000**	0.000
SyRFDRes *var*/*rlx*	**1.000**	0.000	**1.000**	0.000	0.667	0.488	0.600	0.507	**1.000**	0.000	**1.000**	0.000
SyRFDRes *var*/*str*	**1.000**	0.000	**1.000**	0.000	0.615	0.506	0.615	0.506	**1.000**	0.000	**1.000**	0.000

φ: similarity threshold used for $\mathrm{RFD}_{\mathrm{c}}$ discovery; *rlx*: relaxed validation mode; *str*: strict validation mode; *var*: variable per-attribute threshold inferred by Domino [11]. *mean*: mean compliance; *std*: standard deviation.

Table 1: Detailed RFD tuple compliance on both relaxed and strict modes.

3 LLM-BASED GENERATION STRATEGIES

We evaluate LLMs as minority-class generators using eight prompting strategies of increasing reasoning complexity.

Common protocol. Every prompt shares the same skeleton, composed of: *i)* a role instruction framing the model as a tabular data augmentation expert and stating the target minority class; *ii)* schema constraints, listing for each numerical attribute its admissible range, mean, and standard deviation, and for each categorical attribute its admissible values (the ten most frequent levels, five of which are shown in the prompt); *iii)* distribution guidance, reporting the interquartile statistics (Q25, median, Q75) of each numerical attribute; *iv)* five real minority tuples serving as few-shot examples; and *v)* a strict output-format footer requiring a bare CSV with the exact attribute header, no markdown, and no additional text. Tuples are requested in batches of 10 until the minority class matches the majority count. Each raw response is parsed by a format-tolerant CSV extractor, which strips code fences, locates the header line, and attempts multiple separators (comma, semicolon, tab, pipe); the minority label is then enforced on all parsed rows. Statistics are computed once per dataset and reused across batches.

Practical details. Models are queried with temperature 0.7; all other sampling parameters, such as top-p, top-k, or maximum output tokens, are left at their provider defaults, and no fixed random seed is used, so individual runs are not bit-exact reproducible, though aggregate statistics over the 28 benchmarks are stable. Each request is retried up to 3 times with linear backoff, i.e., 5*s*, 10*s*, and 15*s*, and a 120-second timeout per attempt; a batch that still fails after all retries, i.e., empty response, or a response from which no CSV can be extracted, contributes no rows for that round, so a strategy occasionally finishes with fewer synthetic tuples than the

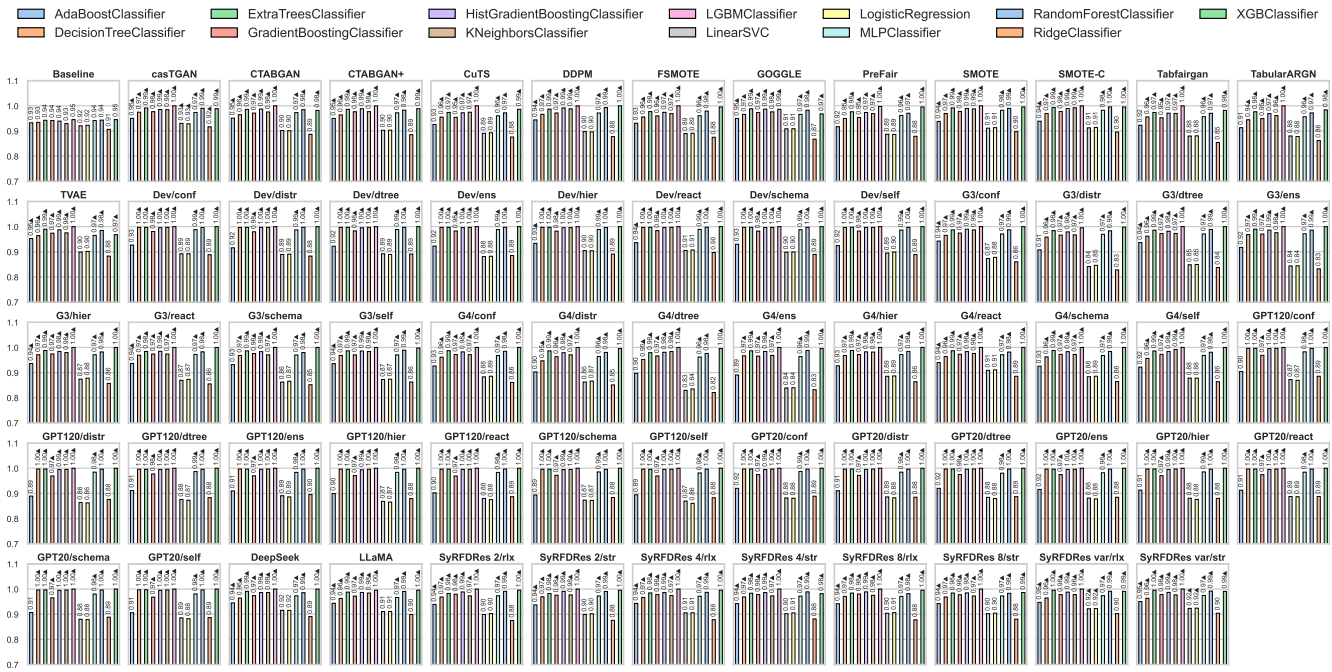


Figure 1: Comparison of accuracy values achieved by 13 classifiers on all datasets augmented with GAN-based, LLM-based, and SyRFDRes techniques against the baseline dataset. The black upward-pointing triangle (▲) above a bar indicates that the model's average accuracy exceeds the baseline's average accuracy for that same classifier.

target diff for datasets on which the model struggles to comply with the output format, this is reflected in the raw generation logs released in the repository. Duplicate or near-verbatim copies of the few-shot examples are not filtered at generation time and count toward the generation budget as any other row, to avoid affecting compliance with tuples that are duplicates of the real data.

Prompting strategies. On top of the shared skeleton, each strategy adds a distinct instruction block:

- *Schema Constraints*: mandates strict adherence to the declared ranges and admissible categorical values, forbidding missing fields; it represents the minimal, constraint-only baseline.
- *Distribution Guidance*: requires the generated values to reflect the observed interquartile ranges, to preserve the inter-attribute correlations visible in the examples, and to avoid verbatim copies.
- *Hierarchical CoT*: guides the model through four explicit reasoning levels before emitting the CSV—schema structure, per-attribute distributions, inter-attribute consistency, and generation.
- *Self-Consistency CoT*: instructs the model to draft the tuples, verify their means and variances against the examples, inspect each row for impossible or contradictory combinations, and emit the corrected CSV only afterwards.
- *ReAct CoT*: decomposes generation into interleaved reasoning and acting steps that extract per-attribute statistics, categorical constraints, and the strongest correlations from the examples before producing the output.
- *Ensemble CoT*: rotates the batches over three perspectives: statistical fidelity, semantic plausibility, and diversity, each instructing the model to prioritize a different quality dimension, so that the final set aggregates complementary viewpoints.
- *Decision-Tree CoT*: asks the model to build a mental decision tree rooted at the minority class and branching on the median of a pivot attribute, then to generate tuples evenly across branches to increase variety.
- *Confidence CoT*: asks the model to score each candidate row on range adherence, categorical coherence, and semantic plausibility, and to retain only rows above a fixed confidence threshold.

Models. The eight strategies are run on five models, Devstral-Small 24B, Gemma-3 12B, Gemma-4 31B, GPT-OSS 20B, GPT-OSS 120B, accessed through the Ollama Cloud platform[1], each queried independently with the retry and timeout policy described above; no fallback across models is used. DeepSeek 3 and Llama 3 are additionally evaluated in a plain zero-shot setting, receiving only the shared skeleton without strategy-specific instructions. The verbatim prompt templates are available in our repository.

4 UTILITY EVALUATION

Figures 1, 2, and 3 provide an overview of the evaluation of augmented datasets when trained on 13 classifiers of various types. These results are compared against a baseline using non-augmented

[1]https://ollama.com/search?c=cloud

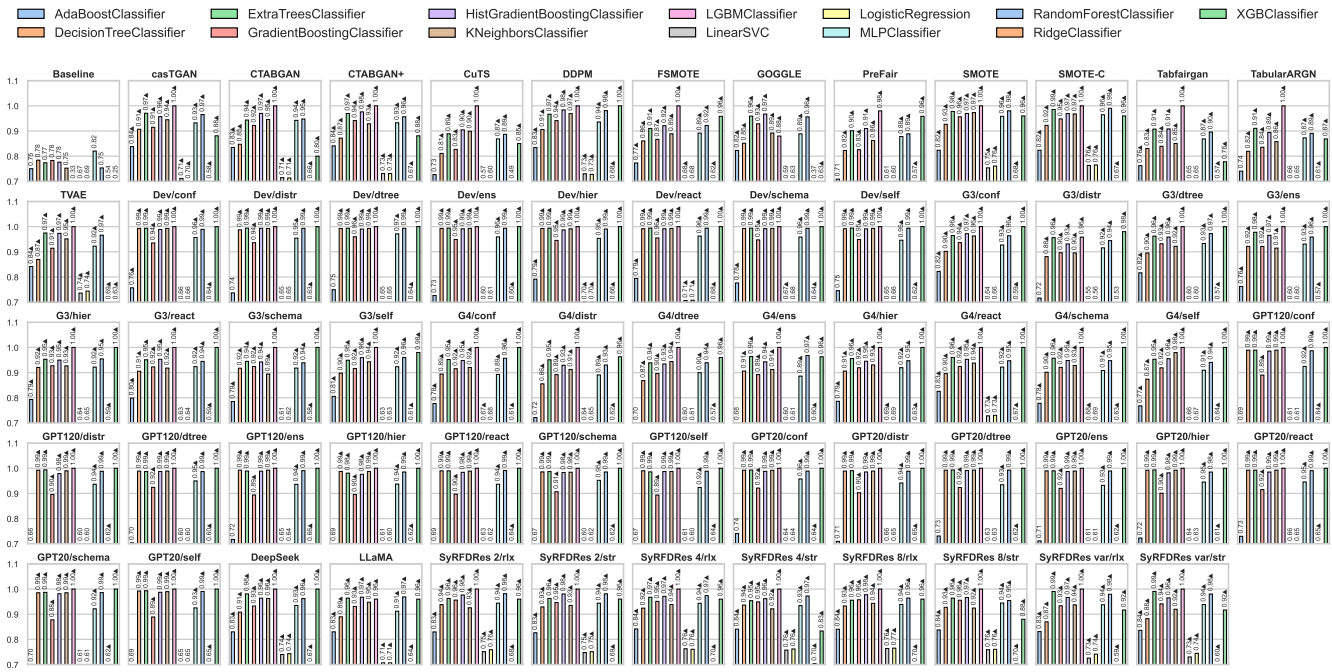


Figure 2: Comparison of F1 values achieved by 13 classifiers on all datasets augmented with GAN-based, LLM-based, and SyRFDRes techniques against the baseline dataset.

datasets and 63 versions of augmented datasets featuring configurations based on GANs and LLMs, such as classical oversampling techniques, GAN-based generators and diffusion models, fairness-aware generators, and five LLMs executed using the eight prompting strategies discussed above.

Accuracy (Fig. 1). Although all datasets are imbalanced, the accuracy achieved on the baseline dataset ranges from 0.91 (Ridge, Logistic Regression) to 0.95 (LGBM, XGBoost). As we can see, most techniques match or marginally exceed these values, and SyRFDRes reaches up to 1.00 for LGBM, ExtraTrees, and XGBoost, on par with the strongest GAN-based and LLM-based competitors. Moreover, as we can see for the three linear classifiers considered, i.e., LinearSVC, Logistic Regression, and Ridge, accuracy decreases with almost every augmentation technique, and SyRFDRes achieves metrics ranging from 59 to 62 out of the 63 competing configurations, depending on the classifier. This is probably due to the fact that a linear model trained on the original dataset can already achieve a deceptively high accuracy by essentially always predicting the majority class. Once trained on a balanced dataset, the same model starts correctly classifying minority-class instances as well, which occasionally leads to misclassifications and therefore a marginal drop in accuracy scores.

F1-score (Fig. 2). On the benchmark dataset, the F1 score drops for boosting classifiers that are not optimized for class imbalance (0.33 for LGBM, 0.25 for XGBoost) and remains moderate for the three linear classifiers (0.54 for Ridge, 0.67 for LinearSVC, 0.69 for logistic regression). As can be seen, each family of boosting techniques bridges this gap for LGBM and XGBoost, but SyRFDRes is the only technique that improves upon the baseline for all 13 classifiers across all eight of its configurations. For the three linear classifiers, however, Fig. 2 reveals a much less uniform picture, in which whether a technique improves or degrades the F1 score relative to the baseline depends heavily on the family of augmentation techniques. Moreover, LLM-based configurations are the most likely to compromise these classifiers, regardless of the underlying model or prompting strategy: 79% of them (33/42) drop below the baseline F1 score for LinearSVC, and 83% (35/42) do so for logistic regression (e.g., `G3/react`, `GPT120/conf`, among many others). GAN-based approaches generally achieve superior performance; nevertheless, they tend to be unstable, with roughly half of them underperforming the baseline for these two classifiers (including GOGGLE, CuTS, PreFair, Tabfairgan, and TabularARGN, among others). SyRFDRes, in contrast, is consistently reliable: all eight of its configurations outperform the baseline for both classifiers, achieving scores of up to 0.76 (LinearSVC) and 0.77 (logistic regression). For Ridge, the situation is mixed, with only three models outperforming the baseline.

Precision and Recall (Fig. 3). Reporting the ratio to the baseline per classifier and technique makes relative gains directly comparable across the grid: markers above the dashed $y = 1$ line denote an improvement in precision or recall over the baseline, markers below denote a degradation. As we can see, SyRFDRes's eight configurations place both the precision and the recall marker at or above 1 for every classifier, with the largest speedups, up to 6×, observed for LGBM and XGBoost, consistent with their very low baseline precision and recall discussed above. Competing techniques are

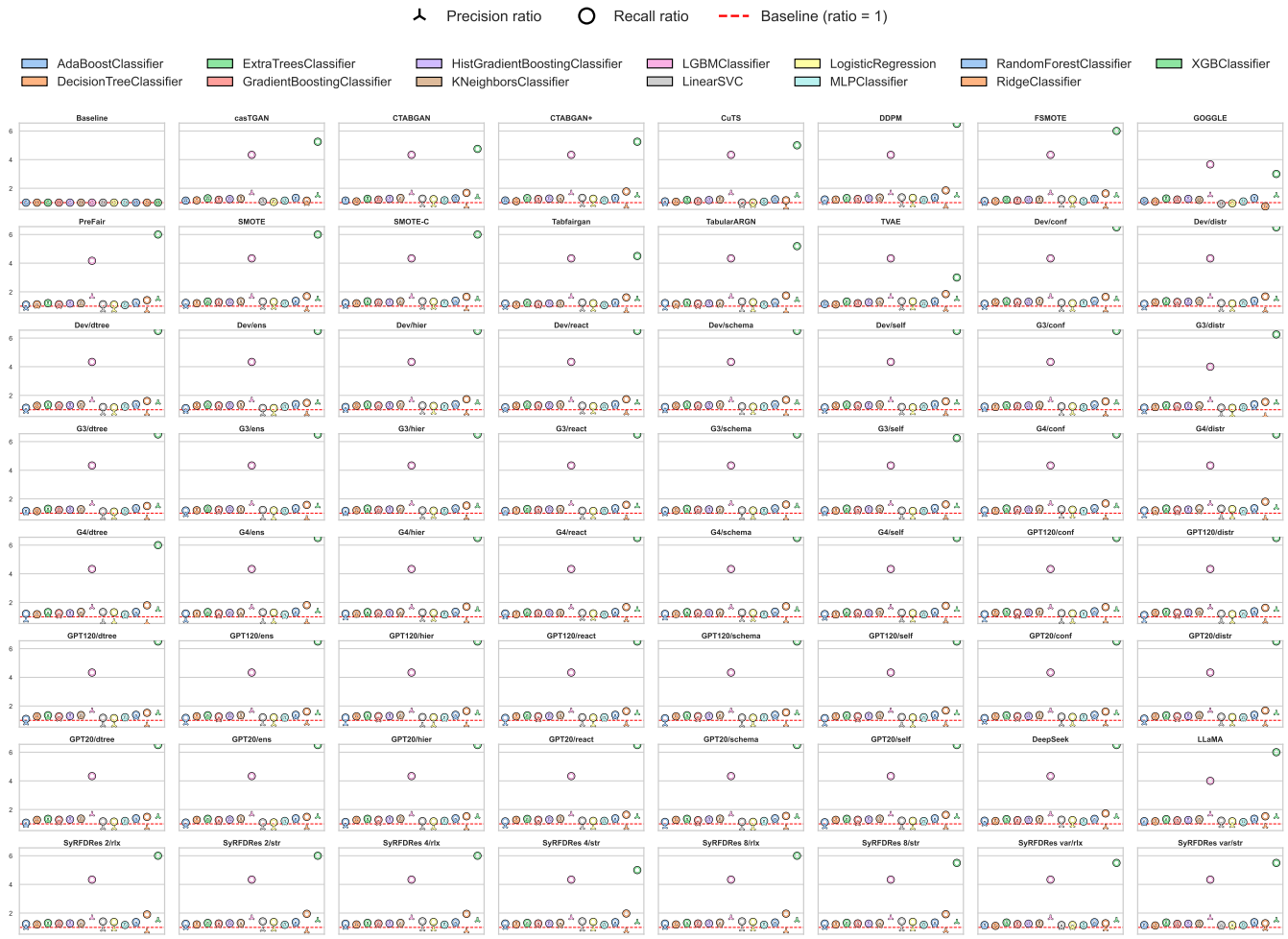


Figure 3: Comparison of average precision and recall for each augmentation technique versus the baseline. Reported values are *speedups*: the ratio between the precision (or recall) of a model trained on data from a given augmentation method and that of the same model trained on the baseline dataset. A value of 2 means the augmented model reaches twice the baseline precision (or recall); a value of 0.5 means worse performance. The dashed red horizontal line at $y = 1$ marks the baseline: markers above it indicate improvement, those below indicate degradation.

markedly less consistent: for the three linear classifiers, several LLM-based configurations place one or both markers below the $y = 1$ line, whereas SyRFDRes's markers remain at or above the line for every classifier throughout all eight configurations, with one exception: for the Ridge classifier, the variable-threshold configurations (`var/rlx`, `var/str`) achieve a more modest speedup than the fixed-threshold ones, consistent with the less score of F1 improvement reported above, though both ratios remain above 1.